

RAG企业知识库项目+LangGraph

第一章：Milvus

1、Milvus 概述

向量是神经网络模型的输出数据格式，可以有效地对信息进行编码，在知识库、语义搜索、检索增强生成（RAG）等人工智能应用中发挥着举足轻重的作用。

Milvus 是一个开源的向量数据库，适合各种规模的人工智能应用。

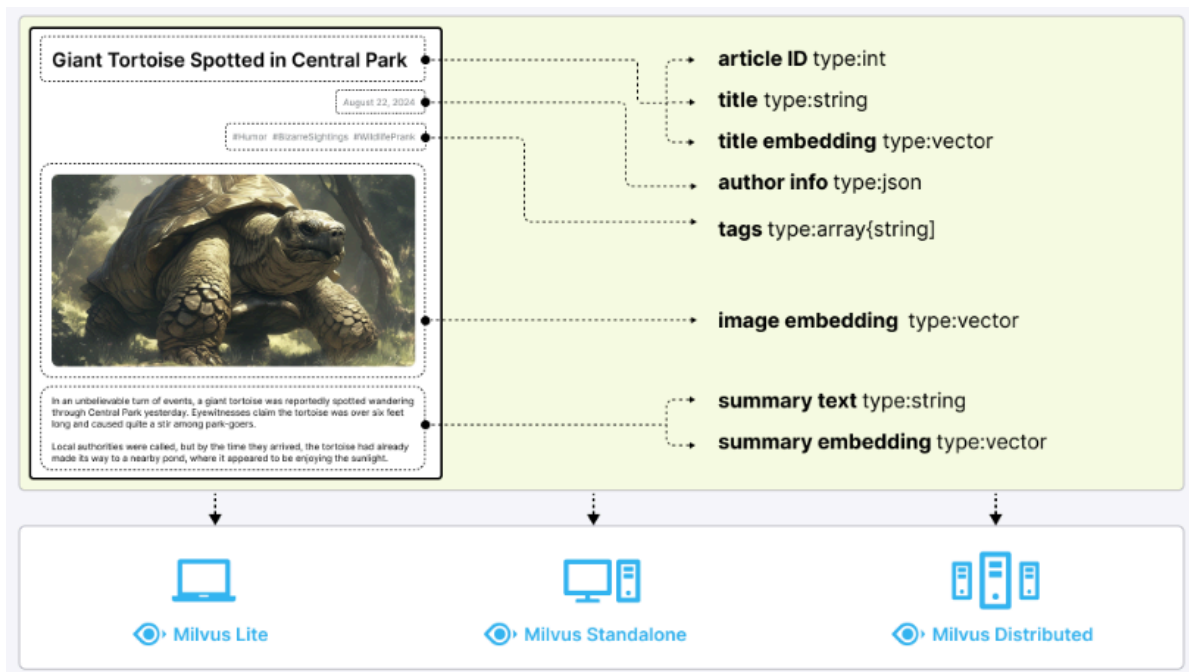
Milvus 是一种高性能、高扩展性的向量数据库，可在从笔记本电脑到大规模分布式系统等各种环境中高效运行。它既可以开源软件的形式提供，也可以云服务的形式提供。

Milvus 是一个开源项目，以 Apache 2.0 许可发布。大多数贡献者都是高性能计算（HPC）领域的专家，擅长构建大型系统和优化硬件感知代码。核心贡献者包括来自 Zilliz、ARM、NVIDIA、AMD、英特尔、Meta、IBM、Salesforce、阿里巴巴和微软的专业人士。

Embeddings 和 Milvus

文本、图像和音频等非结构化数据格式各异，并带有丰富的底层语义，因此分析起来极具挑战性。为了处理这种复杂性，Embeddings 被用来将非结构化数据转换成能够捕捉其基本特征的数字向量。然后将这些向量存储在向量数据库中，从而实现快速、可扩展的搜索和分析。

Milvus 提供强大的数据建模功能，使您能够将非结构化或多模式数据组织成结构化的 Collections。它支持多种数据类型，适用于不同的属性模型，包括常见的数字和字符类型、各种向量类型、数组、集合和 JSON，为您节省了维护多个数据库系统的精力。



Milvus 提供三种部署模式，涵盖各种数据规模--从本地原型到管理数百亿向量的大规模 Kubernetes 集群：

- Milvus Lite 是一个 Python 库，可以轻松集成到您的应用程序中。作为 Milvus 的轻量级版本，它非常适合在开发环境中进行快速原型开发，或在资源有限的边缘设备上运行。
- Milvus Standalone 是单机服务器部署，所有组件都捆绑在一个 Docker 镜像中，方便部署。

- Milvus Distributed 可部署在 Kubernetes 集群上，采用云原生架构，专为十亿规模甚至更大的场景而设计。该架构可确保关键组件的冗余。

Milvus 为何如此快速？

Milvus 从设计之初就是一个高效的向量数据库系统。在大多数情况下，Milvus 的性能是其他向量数据库的 2-5 倍。这种高性能是几个关键设计决策的结果：

硬件感知优化：为了让 Milvus 适应各种硬件环境，我们专门针对多种硬件架构和平台优化了其性能，包括 AVX512、SIMD、GPU 和 NVMe SSD。

高级搜索算法：Milvus 支持多种内存和磁盘索引/搜索算法，包括 IVF、HNSW、DiskANN 等，所有这些算法都经过了深度优化。与 FAISS 和 HNSWLib 等流行实现相比，Milvus 的性能提高了 30%-70%。

C++ 搜索引擎向量数据库性能的 80% 以上取决于其搜索引擎。由于 C++ 语言的高性能、底层优化和高效资源管理，Milvus 将 C++ 用于这一关键组件。最重要的是，Milvus 集成了大量硬件感知代码优化，从汇编级向量到多线程并行化和调度，以充分利用硬件能力。

面向列：Milvus 是面向列的向量数据库系统。其主要优势来自数据访问模式。在执行查询时，面向列的数据库只读取查询中涉及的特定字段，而不是整行，这大大减少了访问的数据量。此外，对基于列的数据的操作可以很容易地进行向量化，从而可以一次性在整个列中应用操作，进一步提高性能。

Milvus 支持的搜索类型

Milvus 支持各种类型的搜索功能，以满足不同用例的需求：

- [ANN 搜索](#)：查找最接近查询向量的前 K 个向量。
- [过滤搜索](#)：在指定的过滤条件下执行 ANN 搜索。
- [范围搜索](#)：查找查询向量指定半径范围内的向量。
- [混合搜索](#)：基于多个向量场进行 ANN 搜索。
- [全文搜索](#)：基于 BM25 的全文搜索。
- [Rerankers](#)：根据附加标准或辅助算法调整搜索结果顺序，完善初始 ANN 搜索结果。
- [获取](#)：根据主键检索数据。
- [查询](#)使用特定表达式检索数据。

人工智能集成

- **Embeddings 模型集成** Embedding 模型将非结构化数据转换为其在高维数据空间中的数字表示，以便您能将其存储在 Milvus 中。目前，PyMilvus (Python SDK) 集成了多个嵌入模型，以便您能快速将数据准备成向量嵌入。
- **Reranker 模型集成** 在信息检索和生成式人工智能领域，Reranker 是优化初始搜索结果顺序的重要工具。PyMilvus 也集成了几种 Rerankers 模型，以优化初始搜索返回结果的顺序。
- **LangChain 和其他人工智能工具集成** 在 GenAI 时代，LangChain 等工具受到了应用程序开发人员的广泛关注。作为核心组件，Milvus 通常在此类工具中充当向量存储。



2、安装Milvus

一共三种：本课程只讲前两种。

Milvus Lite 与 Milvus Standalone 和 Distributed 共享相同的 API，涵盖了向量数据持久化和管理、向量 CRUD 操作、稀疏和密集向量搜索、元数据过滤、多向量和混合搜索（hybrid_search）等大部分功能。

部署模式的选择通常取决于应用程序的开发阶段：

- **用于快速原型开发**

如果您想快速构建原型或用于学习，如检索增强生成（RAG）演示、人工智能聊天机器人、多模态搜索，Milvus Lite 本身或 Milvus Lite 与 Milvus Standalone 的组合都很适合。您可以在笔记本中使用 Milvus Lite 进行快速原型开发，并探索各种方法，如 RAG 中的不同分块策略。您可能希望在小规模生产中部署用 Milvus Lite 构建的应用程序，为真正的用户提供服务，或在更大的数据集（例如超过几百万个向量）上验证想法。Milvus Standalone 是合适的选择。Milvus Lite 的应用逻辑仍可共享，因为所有 Milvus 部署都有相同的客户端应用程序接口。Milvus Lite 中存储的数据也可以通过命令行工具移植到 Milvus Standalone 中。

- **小规模生产部署**

对于早期生产阶段，当项目仍在寻求产品与市场的契合，敏捷性比可扩展性更重要时，Milvus Standalone 是最佳选择。只要有足够的机器资源，它仍然可以扩展到 1 亿向量，同时对 DevOps 的要求也比维护 K8s 集群低得多。

- **大规模生产部署**

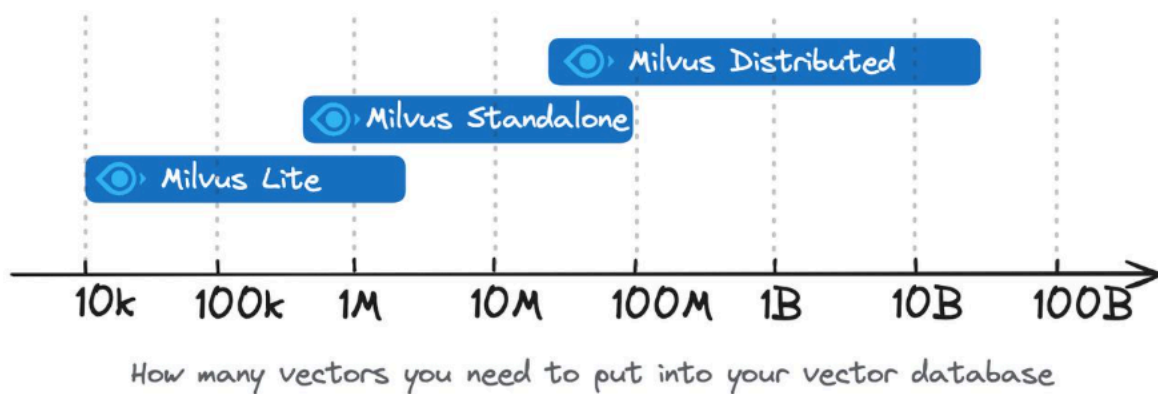
当你的业务快速增长，数据规模超过单台服务器的容量时，是时候考虑 Milvus Distributed 了。你可以继续使用 Milvus Standalone 作为开发或暂存环境，并操作运行 Milvus Distributed 的 K8s 集群。这可以支持你处理数百亿个向量，还能根据你的特定工作负载（如高读取、低写入或高写入、低读取的情况）灵活调整节点大小。

- **边缘设备上的本地搜索**

对于在边缘设备上通过私有或敏感信息进行搜索，您可以在设备上部署 Milvus Lite，而无需依赖基于云的服务来进行文本或图像搜索。这适用于专有文档搜索或设备上对象检测等情况。

Milvus 部署模式的选择取决于项目的阶段和规模。Milvus 为从快速原型开发到大规模企业部署的各种需求提供了灵活而强大的解决方案。

- **Milvus Lite** 建议用于较小的数据集，多达几百万个向量。
- **Milvus Standalone** 适用于中型数据集，可扩展至 1 ~ 10 亿向量。
- **Milvus Distributed** 专为大规模部署而设计，能够处理从一亿到数百亿向量的数据集。



选择适合您使用情况的部署选项

功能	Milvus Lite	Milvus 单机版	分布式 Milvus
SDK / 客户端软件	Python gRPC	Python Go Java Node.js C# RESTful	Python Java Go Node.js C# RESTful
数据类型	密集向量 稀疏向量 二进制向量 布尔值 整数 浮点 VarChar 数组 JSON	密集向量 稀疏向量 二进制向量 布尔型 整数 浮点型 VarChar 数组 JSON	密集向量 稀疏向量 二进制向量 布尔值 整数 浮点 VarChar 数组 JSON
搜索功能	向量搜索 (ANN 搜索) 元数据过滤 范围搜索 标量查询 通过主键获取实体 混合搜索	向量搜索 (ANN 搜索) 元数据过滤 范围搜索 标量查询 通过主键获取实体 混合搜索	向量搜索 (ANN 搜索) 元数据过滤 范围搜索 标量查询 通过主键获取实体 混合搜索
CRUD 操作符	✓	✓	✓
高级数据管理	不适用	访问控制 分区 分区密钥	访问控制 分区 分区密钥 物理资源分组
一致性级别	强	强 有界稳定性 会话 最终	强 有界稳定性 会话 最终

本地运行 Milvus Lite

Milvus Lite 是一个 Python 库，可导入到您的应用程序中。作为 Milvus 的轻量级版本，它非常适合在 Jupyter 笔记本或资源有限的智能设备上运行快速原型。Milvus Lite 支持与 Milvus 其他部署相同的 API。与 Milvus Lite 交互的客户端代码也能与其他部署模式下的 Milvus 实例协同工作。

由于 `milvus-lite` 已包含在 `pymilvus` 2.4.2 或更高版本中，因此可通过 `pip install` 与 `-U` 强制更新到最新版本，`milvus-lite` 会自动安装。

要将 Milvus Lite 集成到应用程序中，请运行 `pip install pymilvus` 进行安装，并使用 `MilvusClient("./demo.db")` 语句实例化一个带有本地文件的向量数据库，以持久化所有数据。

Milvus Lite 目前支持以下环境：

- Ubuntu >= 20.04 (x86_64 和 arm64)
- MacOS >= 11.0 (苹果硅 M1/M2 和 x86_64)

```

3. 终端中长期执行命令请使用screen等工具开后台运行，确保程序不受SSH连接中断影响。https://www.autodl.com/docs/daemon/
root@autodl-container-969947803a-1fda245c: /autodl-tmp/code# pip install pymilvus
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting pymilvus
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/06/4f/169ffe720c628bf89be7770f2ef60f90fd73588c6ceb25d47f1bb
vus-2.5.6-py3-none-any.whl (223 kB)
Requirement already satisfied: setuptools>69 in /root/miniconda3/lib/python3.12/site-packages (from pymilvus) (75.6.
Requirement already satisfied: grpcio<=1.67.1,>=1.49.1 in /root/miniconda3/lib/python3.12/site-packages (from pymilv
Requirement already satisfied: protobuf>=3.20.0 in /root/miniconda3/lib/python3.12/site-packages (from pymilvus) (3.
Requirement already satisfied: python-dotenv<2.0.0,>=1.0.1 in /root/miniconda3/lib/python3.12/site-packages (from py
1)
Collecting ujson>=2.0.0 (from pymilvus)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/04/81/668707e5f217791869b624be4c06fb2473bf97ee33296b18d1cf
-5.10.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (53 kB)
Requirement already satisfied: pandas>=1.2.4 in /root/miniconda3/lib/python3.12/site-packages (from pymilvus) (2.2.3
Collecting milvus-lite>=2.4.0 (from pymilvus)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/44/43/b3f6e9defdf3927b972beac7abe3d5b4a3bdb287e3bad69618e2
s_lite-2.4.12-py3-none-manylinux2014_x86_64.whl (45.2 MB)
45.2/45.2 MB 19.8 MB/s eta 0:00:00
Requirement already satisfied: tqdm in /root/miniconda3/lib/python3.12/site-packages (from milvus-lite>=2.4.0->pymil
Requirement already satisfied: numpy>=1.26.0 in /root/miniconda3/lib/python3.12/site-packages (from pandas>=1.2.4->p
6.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /root/miniconda3/lib/python3.12/site-packages (from pandas>
vus) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /root/miniconda3/lib/python3.12/site-packages (from pandas>=1.2.4->py
.2)
Requirement already satisfied: tzdata>=2022.7 in /root/miniconda3/lib/python3.12/site-packages (from pandas>=1.2.4->
24.2)
Requirement already satisfied: six>=1.5 in /root/miniconda3/lib/python3.12/site-packages (from python-dateutil>=2.8.
2.4->pymilvus) (1.16.0)
Installing collected packages: ujson, milvus-lite, pymilvus
Successfully installed milvus-lite-2.4.12 pymilvus-2.5.6 ujson-5.10.0

```

```

# 导入必要的库
from pymilvus import MilvusClient # Milvus 客户端，用于操作嵌入式向量数据库
import numpy as np # NumPy 库，用于生成随机数和处理数组

# 初始化 Milvus 客户端
# 参数 "./milvus_demo.db" 指定了本地数据库文件的路径
client = MilvusClient("./milvus_demo.db")

# 创建集合 (Collection)
# 集合类似于关系型数据库中的表，用于存储向量和其他字段
client.create_collection(
    collection_name="demo_collection", # 集合名称为 "demo_collection"
    dimension=384 # 向量的维度为 384，表示每个向量是一个长度为 384 的浮点数数组
)

# 准备数据：文档、向量和其他字段
docs = [
    "Artificial intelligence was founded as an academic discipline in 1956.", #
    文档 1
    "Alan Turing was the first person to conduct substantial research in AI.", #
    文档 2
    "Born in Maida vale, London, Turing was raised in southern England.", #
    文档 3
]

# 为每段文本生成一个随机的 384 维向量
# 使用 NumPy 的 `np.random.uniform` 生成范围在 -1 到 1 之间的随机数
vectors = [[np.random.uniform(-1, 1) for _ in range(384)] for _ in
range(len(docs))]

# 将文档、向量、ID 和主题打包成字典格式
# 每个字典包含以下字段：
# - id: 唯一标识符

```

```

# - vector: 向量数据
# - text: 文本内容
# - subject: 主题标签（这里是 "history"）
data = [
    {"id": i, "vector": vectors[i], "text": docs[i], "subject": "history"}
    for i in range(len(vectors))
]

# 将数据插入到集合中
res = client.insert(
    collection_name="demo_collection", # 指定目标集合
    data=data # 要插入的数据列表
)
# 输出插入结果（通常返回成功状态或插入的记录数）
print("Insert result:", res)

# 执行相似性搜索
# 在集合中查找与查询向量最相似的记录
res = client.search(
    collection_name="demo_collection", # 指定目标集合
    data=[vectors[0]], # 查询向量（这里使用了第一个文档的向量）
    filter="subject == 'history'", # 筛选条件：只返回主题为 "history" 的记录
    limit=2, # 返回的最大结果数量（这里是 2 条）
    output_fields=["text", "subject"], # 指定返回的字段（这里返回 "text" 和 "subject"）
)
# 输出搜索结果
print("Search result:", res)

# 执行查询操作
# 根据条件筛选记录（类似于 SQL 查询）
res = client.query(
    collection_name="demo_collection", # 指定目标集合
    filter="subject == 'history'", # 筛选条件：只返回主题为 "history" 的记录
    output_fields=["text", "subject"], # 指定返回的字段（这里返回 "text" 和 "subject"）
)
# 输出查询结果
print("Query result:", res)

# 删除记录
# 根据条件删除记录
res = client.delete(
    collection_name="demo_collection", # 指定目标集合
    filter="subject == 'history'", # 删除条件：删除主题为 "history" 的记录
)
# 输出删除结果
print("Delete result:", res)

```

Milvus 单机版

Milvus Standalone 是单机服务器部署。Milvus Standalone 的所有组件都打包到一个[Docker 镜像](#)中，部署起来非常方便。如果你有生产工作负载，但又不想使用 Kubernetes，那么在内存充足的单机上运行 Milvus Standalone 是一个不错的选择。此外，Milvus Standalone 通过主从复制支持高可用性。

操作系统	软件	备注
macOS 10.14 或更高版本	Docker 桌面	将 Docker 虚拟机 (VM) 设置为至少使用 2 个虚拟 CPU (vCPU) 和 8 GB 初始内存。否则，安装可能会失败。 更多信息，请参阅 在 Mac 上安装 Docker Desktop 。
Linux 平台	- Docker 19.03 或更高版本 - Docker Compose 1.25.1 或更高版本	更多信息，请参阅 安装 Docker Engine 和 安装 Docker Compose 。
已启用 WSL 2 的 Windows	Docker 桌面	我们建议您将绑定挂载到 Linux 容器中的源代码和其他数据存储在 Linux 文件系统中，而不是 Windows 文件系统中。 更多信息，请参见 在 Windows 上安装带有 WSL 2 后端的 Docker Desktop 。

安装 Docker

```
# 安装必要支持
sudo apt install apt-transport-https ca-certificates curl software-properties-common gnupg lsb-release

# 配置 阿里源（推荐使用阿里的gpg KEY）
curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://mirrors.aliyun.com/docker-ce/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
# 更新 源
sudo apt update
sudo apt-get update

# 安装最新版本的 Docker
sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin
# 等待安装完成

# 查看 Docker 版本
sudo docker version

# 查看 Docker 运行状态
sudo systemctl status docker
```

在 Docker 中安装 Milvus

Milvus 提供了一个安装脚本，可将其安装为 docker 容器。该脚本可在[Milvus 存储库](#)中找到。要在 Docker 中安装 Milvus，只需运行

```
# 下载官方提供的安装脚本
curl -sL https://raw.githubusercontent.com/milvus-io/milvus/master/scripts/standalone_embed.sh -o standalone_embed.sh

# 安装Milvus 并且启动
bash standalone_embed.sh start

# 停止Milvus服务
bash standalone_embed.sh stop
# 删除Milvus的所有容器
bash standalone_embed.sh delete
```

你可以访问 Milvus WebUI, 网址是 `http://127.0.0.1:9091/webui/`

```
数据库连接代码: client = MilvusClient(uri="http://1.95.116.112:19530")
```

安装attu

```
docker run -d -p 8000:3000 -e MILVUS_URL=MILVUS_URL:19530 zilliz/attu:v2.5
```

请注意, MILVUS_URL这应该是 Attu Docker 容器可以访问的地址。因此, “127.0.0.1” 或 “localhost” 将不起作用。

第二章：内容加载和切片

1、PDF文件加载和切片

PDF 中的文本通常通过文本框表示。它们也可能包含图像。PDF 解析器可能会执行以下作的某种组合：

- 通过启发式或 ML 推理将文本框聚合为行、段落和其他结构；
- 对图像运行 [OCR](#) 以检测其中的文本；
- 将文本分类为属于段落、列表、表格或其他结构；
- 将文本构建为表格行和列或键值对。

Python中有许多 PDF 解析器集成。有些是简单且相对较低的；其他 API 将支持 OCR 和图像处理，或执行高级文档布局分析。正确的选择将取决于您的需求。

pypdf

它将返回一个 [Document](#) 对象列表 -- 每页一个 -- 在 Document 的属性中包含页面文本的单个字符串。它不会解析图像或扫描的 PDF 页面中的文本。

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader(file_path)
pages = []
async for page in loader.alazy_load():
    pages.append(page)
```

Unstructured

如果您需要对文本进行更精细的分割（例如，分割成不同的段落、标题、表格或其他结构）或需要从图像中提取文本，则以下方法适用。它将返回 Document 对象列表，其中每个对象代表页面上的一个结构。Document 的元数据存储页码和与对象相关的其他信息（例如，如果是 table 对象，它可能会存储 table 行和列）。

`UnstructuredLoader` 是 LangChain 中用于加载非结构化文档（如 PDF、Word、HTML 等）的工具。以下是代码中各个参数的解释：

1. `file_path=pdf_file`
 - 指定要加载的 PDF 文件路径。可以是本地文件路径或 URL。
2. `strategy="hi_res"`
 - 解析策略，决定如何处理文档内容。可选值包括：
 - `"fast"`：快速解析，适合简单文档，但可能忽略复杂布局。
 - `"hi_res"`：高精度解析，适合复杂布局（如多栏、表格、图片），但速度较慢
 - `"auto"`：自动选择策略（默认）。
3. `partition_via_api=True`
 - 是否通过 Unstructured API 进行文档分区（即拆分文档为结构化块）。
 - 若为 `True`，需提供 `api_key` 并依赖网络请求；若为 `False`，则使用本地解析逻辑（需安装额外依赖）。
4. `coordinates=True`
 - 是否保留文本在原始文档中的坐标信息（如位置、边界框）。这对需要精确定位文本的应用（如表格提取）很有用。
5. `api_key='IhWKAZRBmZ14c8tmCsOLabqWIKLJ2e'`
 - Unstructured API 的访问密钥，用于通过云端服务处理文档。若无此密钥，需本地运行分区逻辑

高精度解析：策略为文档布局分析和 OCR 提供支持。

本地构建Unstructured环境

在本地解析需要安装其他依赖项。

Poppler（PDF 分析）

- Linux的: `apt-get install poppler-utils`
- 苹果电脑: `brew install poppler`
- Windows: <https://github.com/oschwartz10612/poppler-windows>

Tesseract（OCR）

- Linux的: `apt-get install tesseract-ocr`
- 苹果电脑: `brew install tesseract`
- Windows: <https://github.com/UB-Mannheim/tesseract/wiki#tesseract-installer-for-windows>

我们还需要安装 PDF 解析器: `unstructured`

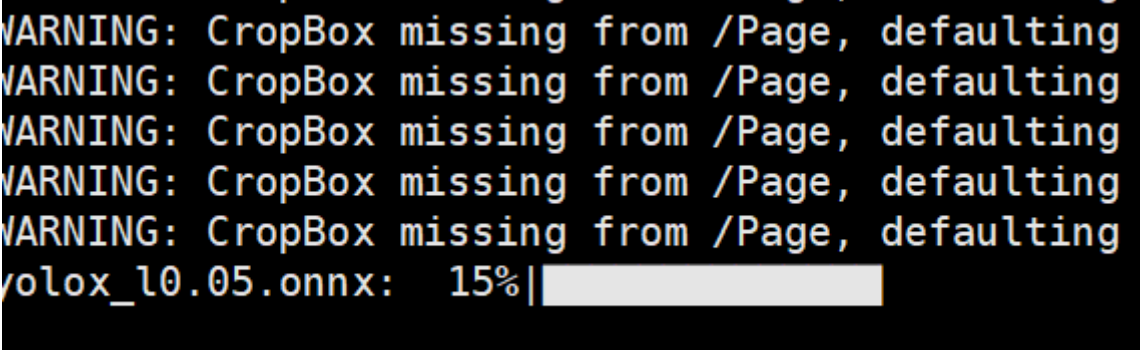
```
%pip install -qu "unstructured[pdf]"
```

```
loader_local = UnstructuredLoader(  
    file_path=file_path,  
    strategy="hi_res",  
)  
docs_local = []  
for doc in loader_local.lazy_load():  
    docs_local.append(doc)
```

安装Langchain的第三方库:

```
pip install langchain langchain-community langchain-unstructured ipython 等等
```

配置HuggingFace镜像站: <https://hf-mirror.com/>



```
WARNING: CropBox missing from /Page, defaulting  
WARNING: CropBox missing from /Page, defaulting  
WARNING: CropBox missing from /Page, defaulting  
WARNING: CropBox missing from /Page, defaulting  
WARNING: CropBox missing from /Page, defaulting  
yolo_x_10.05.onnx: 15%|  
sh://root@connect.westc.anuhub.com:34223
```

1. 安装依赖

```
pip install -U huggingface_hub
```

2. 设置环境变量

Linux

```
export HF_ENDPOINT=https://hf-mirror.com
```

2、Markdown文件加载和切片

需要 Unstructured 包的 UnstructuredMarkdownLoader 对象。首先我们安装它:

```
pip install "unstructured[md]" nltk
```

两种解析md文件的模式:

- `"elements"`: 将Markdown解析为结构化元素 (标题、段落、列表等), 每个元素都带有元数据 (如类型、层级关系等)
- `"single"` (默认): 将整个Markdown文件作为单个文档加载, 不保留结构信息

3、语义切割

在 `SemanticChunker` 中, `breakpoint_threshold_type` 参数用于控制如何确定文本语义分割的阈值, 即何时将文本拆分为不同的块。以下是该参数的详细说明:

参数值及其区别

1. percentile (默认值)
 - 原理：计算所有句子间向量差异的百分位数（如第90百分位），差异大于该百分位数的位置会被拆分。
 - 特点：适用于通用场景，能自适应不同文本的语义分布。
2. standard_deviation
 - 原理：基于句子间差异的标准差设定阈值，差异超过均值加X倍标准差时拆分。
 - 特点：适合数据分布较均匀的文本，对异常值敏感。
3. interquartile
 - 原理：使用四分位距（IQR）确定阈值，差异超过上四分位加一定倍数的IQR时拆分。
 - 特点：对异常值鲁棒，适合长尾分布或噪声较多的文本。
4. gradient
 - 原理：结合百分位数和梯度变化检测语义边界，适用于高度相关或领域特定的文本。
 - 特点：能识别细微的语义变化，适合技术文档或专业领域内容。

应用场景对比

值	适用场景	优点	缺点
percentile	通用文本（新闻、博客）	自适应性强，无需调参	可能忽略局部语义变化
standard_deviation	数据分布均匀的文本（标准化报告）	对均匀分布敏感	易受极端值影响
interquartile	噪声较多的文本（用户评论、社交媒体）	抗噪声能力强	可能过度分割
gradient	专业领域文本（论文、技术文档）	捕捉细微语义变化	计算复杂度较高

第三章：嵌入和存储

1、Embeddings模型

HuggingFace 上的 BGE 模型是最好的开源嵌入模型之一。BGE 模型由北京人工智能研究院（BAAI）创建。是一家从事 AI 研发的私营非营利组织。

BGE-Large（智源研究院）和**GTE-Large**（阿里巴巴）是当前中文RAG领域主流的开源Embedding模型，两者的核心区别与优势如下：

维度	BGE-Large	GTE-Large
研发机构	北京智源研究院	阿里巴巴
模型架构	基于BERT-like架构，优化了RetroMAE预训练算法	基于BERT架构，结合LLM（如Qwen）训练
语言支持	中英文（中文优化更显著）	中英文（多语言扩展性更强）
序列长度	默认512 tokens（BGE-M3支持8192）	默认512 tokens（GTE-Multilingual支持8k）
向量维度	1024维	1024维
训练数据	C-MTP（1亿+中文文本对）	大规模多语言语料（含2200万+中文句对）
开源协议	MIT（免费商用）	部分版本商用需授权

https://huggingface.co/BAAI/bge-large-zh-v1.5

 **Hugging Face**

Search models, datasets, users...

Models

BAAI/bge-large-zh-v1.5 like 506 Follow Beijing Academy of Arti... 1.55k

Feature Extraction sentence-transformers PyTorch Transformers Chinese bert sentence-simil

Model card Files and versions Community 33

FlagEmbedding

[Model List](#) | [FAQ](#) | [Usage](#) | [Evaluation](#) | [Train](#) | [Contact](#) | [Citation](#) | [License](#)

https://huggingface.co/BAAI/bge-large-zh-v1.5

Using Langchain

You can use in langchain like this: bge

```
from langchain.embeddings import HuggingFaceBgeEmbeddings
model_name = "BAAI/bge-large-en-v1.5"
model_kwargs = {'device': 'cuda'}
encode_kwargs = {'normalize_embeddings': True} # set True to compute cosine simil
model = HuggingFaceBgeEmbeddings(
    model_name=model_name,
    model_kwargs=model_kwargs,
    encode_kwargs=encode_kwargs,
    query_instruction="为这个句子生成表示以用于检索相关文章："
)
model.query_instruction = "为这个句子生成表示以用于检索相关文章："
```

```
pip install --upgrade --quiet sentence_transformers
```

```
from langchain.embeddings import HuggingFaceBgeEmbeddings
model_name = "BAAI/bge-large-zh-v1.5"
model_kwargs = {'device': 'cuda'}
encode_kwargs = {'normalize_embeddings': True} # set True to compute cosine
similarity
model = HuggingFaceBgeEmbeddings(
    model_name=model_name,
    model_kwargs=model_kwargs,
    encode_kwargs=encode_kwargs,
    query_instruction="为这个句子生成表示以用于检索相关文章："
)
model.query_instruction = "为这个句子生成表示以用于检索相关文章："
```

在 `HuggingFaceBgeEmbeddings` 中，`normalize_embeddings` 参数通常只接受布尔值（`True` 或 `False`），用于决定是否对生成的嵌入向量进行归一化处理。具体来说：

- `True`：生成的嵌入向量会被归一化为单位向量。这意味着每个嵌入向量的 L2 范数（欧几里得长度）将被缩放到 1。
- `False`：生成的嵌入向量将保持原始的数值，不进行归一化处理。
- 优点：
 - **提高相似度计算的稳定性**：在许多应用场景中，如余弦相似度计算，归一化后的向量可以避免因向量长度不同而导致的相似度偏差，使相似度计算更加专注于向量的方向而非长度。
 - **一致性**：在某些情况下，归一化可以确保不同批次或不同模型生成的嵌入向量在同一尺度上，便于比较和整合。

2、BM25 详解

BM25 (Best Matching 25) 是一种 **基于统计的稀疏检索算法**，是信息检索 (Information Retrieval, IR) 领域最经典的排序函数之一。它是对传统 **TF-IDF (词频-逆文档频率)** 的改进，能够更合理地衡量文档与查询的相关性，广泛应用于搜索引擎 (如 Elasticsearch、Lucene) 和问答系统。

1. BM25 的核心思想

BM25 的核心是计算**查询 (Query)** 和**文档 (Document)** 之间的相关性得分，主要考虑以下因素：

1. **词频 (Term Frequency, TF)**：查询词在文档中出现的频率（越高越相关）。
2. **逆文档频率 (Inverse Document Frequency, IDF)**：查询词在整个语料库中的稀有程度（越稀有越重要）。
3. **文档长度归一化 (Document Length Normalization)**：避免长文档因词频高而占据优势。

BM25 的公式比 TF-IDF 更精细，引入了可调参数 (k_1 和 b)，使其对不同数据集更鲁棒。

BM25 对查询 Q 和文档 D 的相关性评分公式如下：

$$\text{BM25}(Q, D) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

其中：

- q_i ：查询 Q 中的第 i 个词。
- $f(q_i, D)$ ：词 q_i 在文档 D 中的词频 (TF) 。
- $\text{IDF}(q_i)$ ：词 q_i 的逆文档频率 (IDF) ，通常计算为：

$$\text{IDF}(q_i) = \log \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

- N ：语料库中的文档总数。
- $n(q_i)$ ：包含词 q_i 的文档数量。
- $|D|$ ：文档 D 的长度 (词数) 。
- avgdl ：语料库中所有文档的平均长度。
- k_1 和 b ：可调参数 (通常 $k_1 \in [1.2, 2.0]$, $b \in [0.5, 0.8]$) 。

特性	TF-IDF	BM25
词频 (TF) 处理	线性增长 (可能偏向长文档)	非线性饱和 (避免过度依赖高频词)
文档长度归一化	无	引入参数 b ，惩罚长文档
参数可调性	固定	可调整 k_1 (控制词频影响) 和 b (控制长度惩罚)
实际效果	简单但可能过拟合	更鲁棒，适用于不同数据集

BM25 的优势：

- **更合理的词频饱和**：高频词不会无限增加分数 (TF 部分用 $\frac{(k_1+1) \cdot f}{f+k_1}$ 限制) 。
- **文档长度自适应**：通过 b 调整，避免长文档因词多而占优。
- **计算高效**：适合大规模检索 (如 Elasticsearch 默认使用 BM25) 。

3、构建向量知识库

需要安装：pip install langchain-milvus

```
from langchain_milvus import Milvus, BM25BuiltInFunction
from langchain_openai import OpenAIEmbeddings

vectorstore = Milvus.from_documents(
    documents=docs,
    embedding=OpenAIEmbeddings(),
    builtin_function=BM25BuiltInFunction(), # output_field_names="sparse"),
    vector_field=["dense", "sparse"],
    connection_args={
        "uri": URI,
    },
    consistency_level="Strong",
    drop_old=True,
)
```

当您使用 `BM25BuiltInFunction` 时，请注意全文检索在 Milvus Standalone 和 Milvus Distributed 中可用，但在 Milvus Lite 中不可用。

在上面的代码中，我们定义了 `BM25BuiltInFunction` 的一个实例，并将其传递给 `Milvus` 对象。`BM25BuiltInFunction` 是一个轻量级封装类。`Function` 的轻量级封装类。我们可以将它与 `OpenAIEmbeddings` 一起使用，初始化密集+稀疏混合搜索 Milvus 向量存储实例。

`BM25BuiltInFunction` Milvus "不要求客户端传递语料或训练，所有这些都在 Milvus 服务器端自动处理，因此用户无需关心任何词汇和语料。

Milvus 支持的四种一致性级别

级别	描述	适用场景
"Strong"	写入后立即可读，所有查询都能看到最新数据（最高一致性）。	金融交易、实时计费等严格要求数据一致的场景。
"Session"	保证当前客户端会话内能读到自己的写入（其他会话可能有延迟）。	大多数读写平衡的场景（默认级别）。
"Bounded"	允许一定时间（如 5 秒）内的数据延迟，之后保证一致性。	可容忍短暂延迟的高吞吐场景。
"Eventually"	不保证立即一致性，最终会同步（延迟最低，性能最高）。	日志分析、离线计算等对实时性要求低的场景。

度量类型	索引类型
• 欧氏距离 (L2) • 内积 (IP) • 余弦相似度 (COSINE)	• 平面 • IVF_FLAT • IVF_SQ8 • IVF_PQ • GPU_IVF_FLAT • GPU_IVF_PQ • HNSW • DISKANN

索引类型	算法	优点	缺点
HNSW	图搜索	查询快，精度高	内存占用高
IVF_FLAT	倒排+聚类	内存低，适合大规模	需训练，精度略低
IVF_SQ8	量化版 IVF	内存极低	精度损失明显

第四章：高级RAG检索

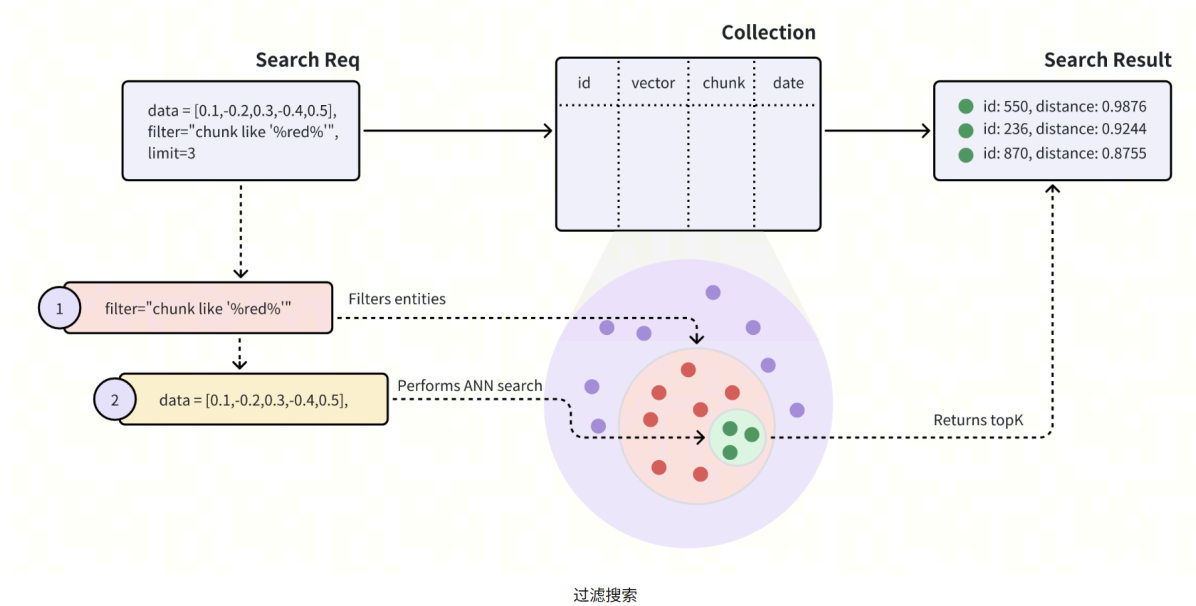
1、近似近邻（ANN）搜索

近似近邻（ANN）搜索以记录向量嵌入排序顺序的索引文件为基础，根据接收到的搜索请求中携带的查询向量查找向量嵌入子集，将查询向量与子群中的向量进行比较，并返回最相似的结果。

2、过滤搜索

ANN 搜索能找到与指定向量嵌入最相似的向量嵌入。但是，搜索结果不一定总是正确的。您可以在搜索请求中包含过滤条件，这样 Milvus 就会在进行 ANN 搜索前进行元数据过滤，将搜索范围从整个 Collections 缩小到只搜索符合指定过滤条件的实体。

如果 Collections 同时包含向量嵌入及其元数据，您可以在 ANN 搜索之前过滤元数据，以提高搜索结果的相关性。Milvus 收到携带过滤条件的搜索请求后，会将搜索范围限制在符合指定过滤条件的实体内。



如上图所示，搜索请求携带 `chunk like % red %` 作为过滤条件，表明 Milvus 应在 `chunk` 字段中包含 `red` 的所有实体内进行 ANN 搜索。具体来说，Milvus 会执行以下操作。

- 过滤符合搜索请求中过滤条件的实体。
- 在过滤后的实体中进行 ANN 搜索。
- 返回前 K 个实体。

基本操作符

Milvus 支持几种用于过滤数据的基本操作符。

- **比较操作符**：`==`, `!=`, `>`, `<`, `>=`, 和 `<=` 允许基于数字、文本或日期字段进行筛选。
- **范围过滤器**：`IN` 和 `LIKE` 可帮助匹配特定的值范围或集合。
- **算术操作符**：`+`, `-`, `*`, `/`, `%`, 和 ``` 用于涉及数字字段的计算。
- **逻辑操作符**：`AND`, `OR`, 和 `NOT` 或 `&&`, `||`, `~`, `!` 将多个条件组合成复杂的表达式。

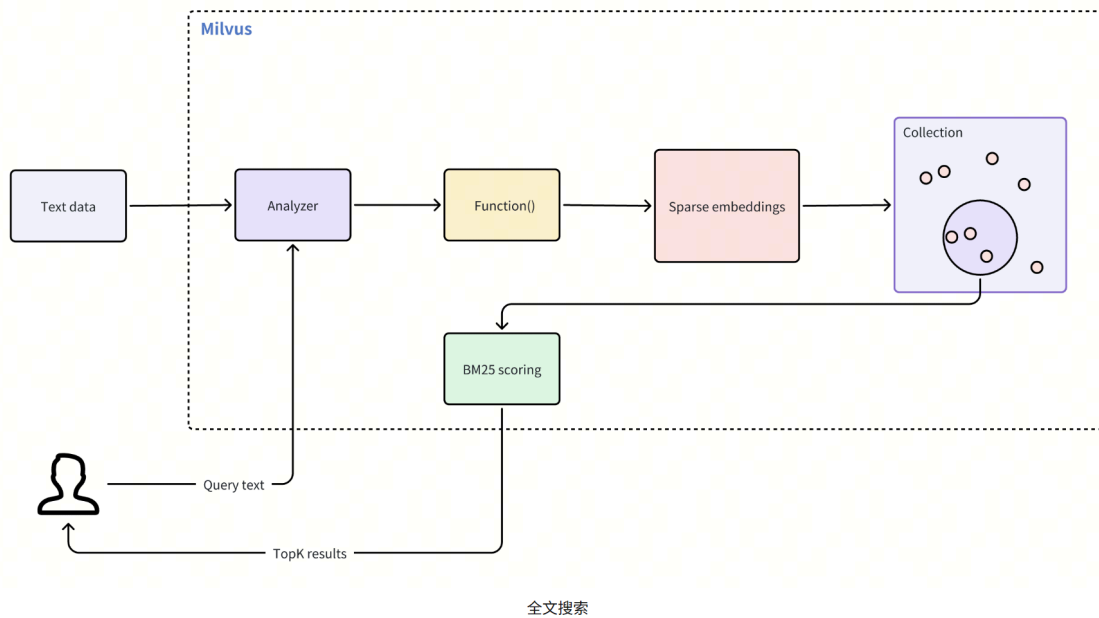
```
filter = "age > 25 and city in ['北京', '上海']"
```

3、全文搜索 (BM25)

全文搜索是一种在文本数据集中检索包含特定术语或短语的文档，然后根据相关性对结果进行排序的功能。该功能克服了语义搜索可能会忽略精确术语的局限性，确保您获得最准确且与上下文最相关的结果。此外，它还通过接受原始文本输入来简化向量搜索，自动将您的文本数据转换为稀疏嵌入，而无需手动生成向量嵌入。

该功能使用 BM25 算法进行相关性评分，在检索增强生成 (RAG) 场景中尤为重要，它能优先处理与特定搜索词密切匹配的文档。

全文搜索在 Milvus Standalone 和 Milvus Distributed 中可用，但在 Milvus Lite 中不可用



要使用全文搜索，请遵循以下主要步骤。

1. 创建 Collections：设置一个带有必要字段的 Collections，并定义一个将原始文本转换为稀疏嵌入的函数。
2. 插入数据：将原始文本文档插入 Collections。
3. 执行搜索：使用查询文本搜索你的 Collections 并检索相关结果。

```
search_params = {
    'params': {'drop_ratio_search': 0.2}, # Proportion of small vector values to
    ignore during the search
}

client.search(
    collection_name='demo',
    data=['whats the focus of information retrieval?'],
    anns_field='sparse',
    limit=3,
    search_params=search_params
)
```

4、混合搜索

混合搜索指的是一种同时进行多个 ANN 搜索、对这些 ANN 搜索的多组结果进行 Rerankers 并最终返回一组结果的搜索方法。使用混合搜索可以提高搜索精度。混合搜索最常用于稀疏密集向量搜索和多模态搜索等场景。

[MilvusCollectionHybridSearchRetriever](#) 是使用 Milvus 和 LangChain 进行混合搜索的另一种实现，**即将被弃用**。

应用场景

混合搜索适用于以下两种情况。

稀疏-密集向量搜索

不同类型的向量可以表示不同的信息，使用各种嵌入模型可以更全面地表示数据的不同特征和方面。例如，对同一个句子使用不同的 Embeddings 模型，可以生成表示语义的密集向量和表示句子中词频的稀疏向量。

- **稀疏向量：**稀疏向量的特点是其向量维度高，存在很少的非零值。这种结构使其特别适合传统的信息检索应用。在大多数情况下，稀疏向量中使用的维数对应于一种或多种语言中的不同词块。每个维度都有一个值，表示该标记在文档中的相对重要性。这种布局对于涉及文本匹配的任务非常有利。
- **密集向量：**密集向量是从神经网络中衍生出来的嵌入。当排列成有序数组时，这些向量能捕捉到输入文本的语义本质。需要注意的是，稠密向量并不局限于文本处理；它们还广泛应用于计算机视觉，以表示视觉数据的语义。这些稠密向量通常由文本 Embeddings 模型生成，其特点是大部分或所有元素都非零。因此，密集向量对于语义搜索应用特别有效，因为即使在没有精确文本匹配的情况下，它们也能根据向量距离返回最相似的结果。这种功能可以获得更细致入微、更能感知上下文的搜索结果，通常可以捕捉到基于关键词的方法可能忽略的概念之间的关系。

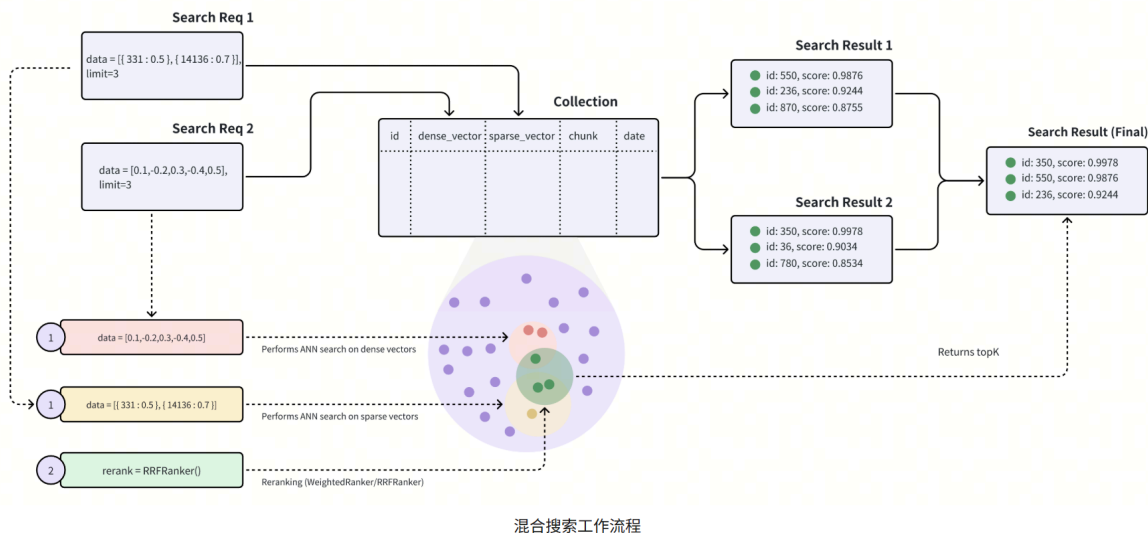
多模式搜索

多模态搜索是指跨多种模态（如图像、视频、音频、文本等）对非结构化数据进行相似性搜索。例如，一个人可以用指纹、声纹和面部特征等多种数据模式来表示。混合搜索支持同时进行多种搜索。例如，用相似的指纹和声纹搜索一个人。

工作流程

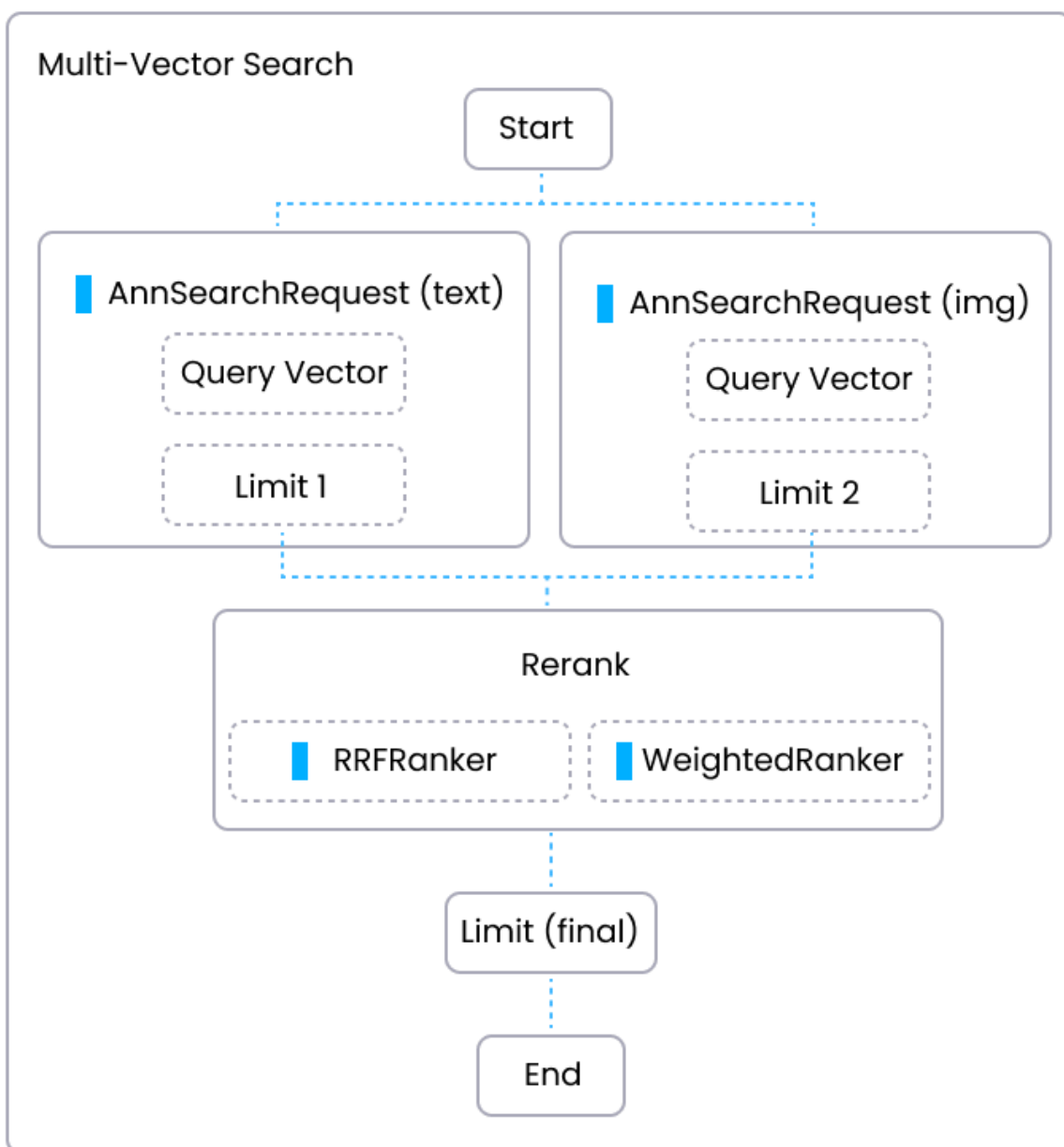
混合搜索的主要工作流程如下。

1. 通过BERT和Transformers 等嵌入模型生成密集向量。
2. 通过BM25、BGE-M3、SPLADE 等嵌入模型生成稀疏向量。
3. 创建 Collections 并定义 Collections Schema，其中包括密集向量场和稀疏向量场。
4. 将稀疏密集向量插入上一步刚刚创建的 Collections 中。
5. 进行混合搜索：稠密向量上的 ANN 搜索将返回一组前 K 个最相似的结果，稀疏向量上的文本匹配也将返回一组前 K 个结果。
6. 归一化：对两组 K 强结果的得分进行归一化，将得分转换为 [0,1] 之间的范围。
7. 选择适当的 Rerankers 策略，对两组 Top-K 结果进行合并和重排，最终返回一组 Top-K 结果。



5、重新排名

下图展示了在 Milvus 中执行混合搜索的过程，并强调了重排在此过程中的作用。



混合搜索中的重新排序是一个关键步骤，它可以整合来自多个向量场的结果，确保最终输出结果具有相关性并能准确排出优先级。目前，Milvus 提供以下重新排序策略：

- WeightedRanker**: 这种方法通过计算来自不同向量搜索的得分（或向量距离）的加权平均值来合并结果。它根据每个向量场的重要性分配权重。
- RRFRanker**: 这种策略根据结果在不同向量列中的排名来合并结果。

加权评分 (WeightedRanker)

WeightedRanker 策略根据每个向量字段的重要性，为每个向量检索路径的结果分配不同的权重。当每个向量字段的重要性不同时，就会应用这种 Rerankers 策略，这样就可以通过给某些向量字段分配更高的权重，使其比其他向量字段更受重视。例如，在多模态搜索中，文本描述可能比图像中的颜色分布更重要。

互易等级融合 (RRFRanker)

RRF 是一种数据融合方法，它根据排名的倒数来组合排名列表。它是一种平衡各向量场影响的有效方法，尤其是在没有明确的重要性优先顺序时。这种策略通常用于想要对所有向量场给予同等考虑，或对每个场的相对重要性存在不确定性时。

RRF 的基本流程如下：

- 在检索过程中收集排名**：检索器跨多个向量字段检索并对结果进行排序。
- 排名融合**：RRF 算法对每个检索器的排名进行权衡和合并。计算公式如下

$$\text{RRF_score}(d) = \sum_{i=1}^N \frac{1}{k + \text{rank}_i(d)}$$

这里， N 代表不同检索路径的数量， $\text{rank}_i(d)$ 是第 i 个检索器检索到的文档 d 的排名位置， k 是平滑参数，通常设置为 60。

- 综合排名**：根据综合得分对检索结果重新排序，得出最终结果。

RRF 允许在不指定明确权重的情况下平衡各领域的影响。在最终排名中，多个字段一致同意的最匹配结果将被优先排序。

在 RRF (Reciprocal Rank Fusion) 算法中，**参数 k 是平滑因子**，用于控制排名对最终得分的影响程度。以下是 $k=60$ 和 $k=100$ 的核心区别：

参数	高排名敏感性	结果多样性	适用场景
k=60	更强（激进）	较低	需优先保证Top结果精准（如问答系统）
k=100	较弱（保守）	较高	需平衡多样性与相关性（如推荐系统）

示例说明

假设两路检索结果如下：

- 稠密向量检索**：文档A排名第1，文档B排名第5。
- 稀疏向量检索**：文档A排名第3，文档B排名第1。

k=60 的得分计算：

- 文档A: $1/(60+1)+1/(60+3)\approx 0.032$
- 文档B: $1/(60+5)+1/(60+1)\approx 0.034 \rightarrow$ 文档B胜出

k=100 的得分计算：

- 文档A: $1/(100+1)+1/(100+3)\approx 0.020$
- 文档B: $1/(100+5)+1/(100+1)\approx 0.019 \rightarrow$ 文档A胜出

可见，**k=60** 更倾向向稀疏向量检索的高排名项（文档B），而 **k=100** 平衡了双方权重。

nprobe=10

功能说明

- **作用场景：**用于Milvus/FAISS等向量数据库的近似最近邻（ANN）搜索，属于IVF（Inverted File）类索引的核心参数。
- **定义：****nprobe** 控制搜索时访问的聚类中心（桶）数量。例如，若索引有1024个聚类中心（**nlist=1024**），**nprobe=10** 表示仅搜索距离查询向量最近的10个聚类中心内的向量。
- **影响：**
 - **精度：**值越大（如 **nprobe=128**），搜索范围越广，召回率越高，但计算量增加。
 - **性能：**值越小（如 **nprobe=1**），搜索速度越快，但可能遗漏相关结果。

```
# 数值比较
filter = "price < 100"
filter = "rating >= 4.5"

# 字符串匹配
filter = "category == '科技'"
filter = "author in ['Alice', 'Bob']"

# 逻辑组合
filter = "price < 100 AND category == '科技'"
filter = "status == 1 OR is_featured == true"
```

第五章：自我评测的RAG

1、Corrective RAG（自我纠正的RAG）

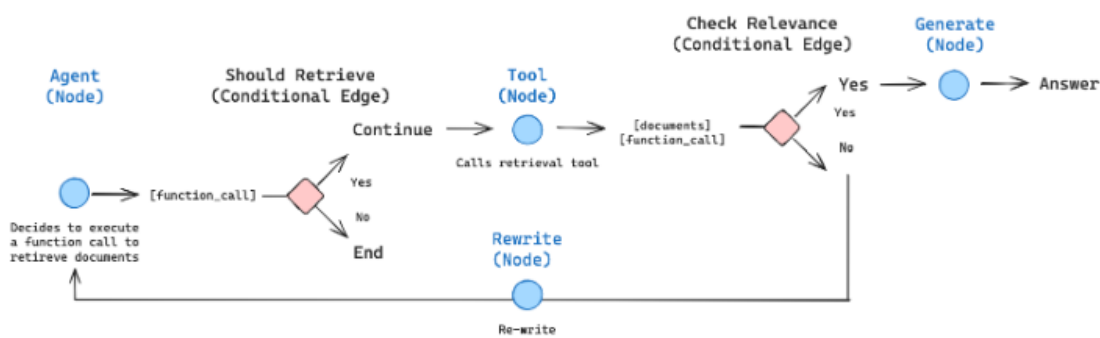
核心概念

Corrective RAG 是 RAG（检索增强生成）的改进范式，通过引入 **实时反馈与纠正机制**，在生成过程中动态检测和修正错误，显著提升输出的准确性和可靠性。其核心思想是：

- **“生成-评估-纠正”闭环：**在传统RAG的单次检索生成基础上，增加对生成内容的实时验证与迭代优化。
- **多粒度纠错：**从事实准确性、逻辑一致性、上下文连贯性等维度进行自我修正。

2. 与传统 RAG 的区别

特性	传统 RAG	Corrective RAG
错误处理	依赖检索库质量，无主动纠错	动态检测并修正生成中的错误
生成流程	单向（检索→生成）	循环（生成→评估→纠正→再生成）
反馈信号	无或人工反馈	自动化验证（如事实核查、逻辑验证）
输出可靠性	易受检索噪声或LLM幻觉影响	通过多次迭代降低错误率



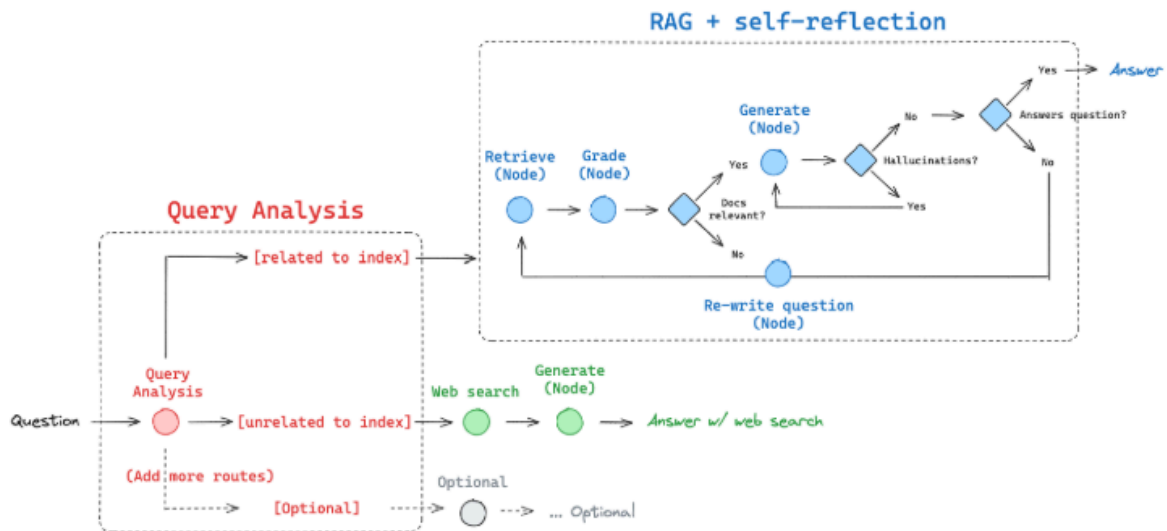
2、Adaptive RAG（自适应RAG）

Adaptive RAG 是一种动态调整检索和生成策略的 RAG 增强范式，旨在根据 **输入查询的复杂性、上下文需求和数据分布**，智能优化检索范围、生成方式和资源分配。其核心思想是：

- **动态适应性**：不再采用固定的检索-生成流程，而是根据任务需求调整策略。
- **资源效率**：在简单查询时减少计算开销，在复杂查询时增强检索和推理能力。
- **上下文感知**：结合用户历史交互、领域知识等优化检索和生成。

与传统 RAG 的区别

特性	传统 RAG	Corrective RAG
错误处理	依赖检索库质量，无主动纠错	动态检测并修正生成中的错误
生成流程	单向（检索→生成）	循环（生成→评估→纠正→再生成）
反馈信号	无或人工反馈	自动化验证（如事实核查、逻辑验证）
输出可靠性	易受检索噪声或LLM幻觉影响	通过多次迭代降低错误率



核心区别对比

维度	Adaptive RAG	Corrective RAG
优化目标	动态调整检索策略	检测并修正生成答案的错误
关键模块	路由机制、置信度判断	验证模块、迭代修正
适用场景	查询复杂度差异大的场景	高精度、低容错的场景
处理时机	检索前决策	生成后修正
计算开销	可能减少检索次数	可能增加验证和修正的开销